BDE Presentation Script — 3 Speakers

Oct 6, 2026 · @Kushal Mohan

How to use this script

The report is split into three parts that follow the marking scheme, about 2½ minutes per speaker.

Speaker	Covers	Rubric line	Time
1 — Kriti	Both problem statements, why both, model choice for each	Model Selection & Justification (1 mark)	~2:30
2 — Kushal	Platform, pipeline, features, training code, clustering code	Coding & Implementation (2 marks)	~2:30
3 — Gargi	Classification results, the 92% twist and fix, clustering results, limitations	Results, Analysis & Inference (1 mark)	~2:45

Plain paragraphs are what you **say out loud**. Lines in grey quote boxes starting with **Note** are for your own understanding only — do not read them aloud. Each note explains a term in one or two lines.

Speaker 1 — Kriti: the problems and model choice (~2:30)

Opening

Good morning. We are Kriti, Kushal and Gargi, and our project is a Log Analytics System for a Web Server. Every time someone visits a website, the server writes a line into a log file. Big websites write millions of these lines every day. Hidden inside them are attacks and unusual visitors. Our project finds both, using two techniques: classification and clustering.

> **Note — Log file:** a text file where the web server records every request: who asked, what page, when, and whether it worked.

> **Note — Classification vs clustering:** classification learns from labelled examples ("this is an attack") and predicts labels for new data. Clustering has no labels; it groups similar things together on its own.

Problem 1 — Classification

Most security tools today use signatures — a list of known bad words, like the SQL command UNION SELECT. The problem is that a brand-new attack, called a zero-day, does not use words on that list, so it gets through. Machine learning can help, but on huge data streams it brings two problems: it can be slow, and turning text into numbers usually creates tens of thousands of columns.

So our goal was a classifier that sorts every request into one of five classes — normal, SQL injection, cross-site scripting, command injection, and path traversal — is fast, and uses only 42 features.

> **Note — the four attacks:** *SQL injection* sneaks database commands into a form field. *Cross-site scripting (XSS)* sneaks JavaScript into a page. *Command injection* sneaks operating-system commands in. *Path traversal* uses `../` to reach files it shouldn't, like `/etc/passwd`.

> **Note — Feature:** one number that describes the input, for example "length of the request". The model only ever sees these numbers.

> **Note — TF-IDF / n-grams:** common ways to turn text into numbers by counting words or letter-chunks. They create huge, mostly-empty tables, which is why we avoided them.

Problem 2 — Clustering

Our second problem is the opposite. Rule-based log analysis can only find what someone already thought to write a rule for. We wanted to group website visitors by how they behave, with no labels at all, and see who behaves unlike everyone else.

These two halves need each other. Classification can only find attacks that someone has already labelled. Clustering can spot something brand new, but cannot tell you what it is called. Together, each covers the other's blind spot.

Model choice — Classification

We did not pick a model by reputation. We trained eight classifiers on the same 80/20 split and measured all of them. Our faculty asked us not to use very common models like Random Forest, so tree models were kept only as a comparison.

> **Note — 80/20 split:** 80% of the data is used to train the model, 20% is hidden and used only to test it.

> **Note — Random Forest / Decision Tree:** models that make predictions by asking a chain of yes/no questions. Very popular, which is why they were excluded.

The winner was a Multilayer Perceptron. It had the highest score, macro-F1 of 0.9984, the best result on every class, and it was the fastest — 20.3 milliseconds per thousand requests. It even beat the Random Forest we were told not to use, so the rule cost us nothing.

> **Note — Multilayer Perceptron (MLP):** a small neural network. Ours has 4 layers: 42 inputs → 32 → 16 → 5 outputs (one per class). Each layer combines the numbers from the layer before, which lets it learn combinations like "lots of punctuation AND high randomness".

> **Note — Macro-F1:** a score from 0 to 1 that averages performance across all classes equally, so rare attack types count as much as common ones.

The others lost for clear reasons. Factorization Machines were nearly as accurate but eleven times slower. Logistic Regression can only draw straight-line boundaries, and its path-traversal score fell to 72.9%. Naive Bayes assumes every feature is independent, which is false here, so it scored lowest.

> **Note — Factorization Machines:** a model that learns hidden "factors" to capture how pairs of features interact; often used in recommendation systems.

> **Note — Logistic Regression:** the simplest classifier; draws a straight line (or flat surface) between classes.

> **Note — Naive Bayes:** a probability-based model that treats each feature as if it had nothing to do with the others.

Model choice — Clustering

For clustering we chose a Gaussian Mixture Model with three groups. Bots are a tiny group, about 0.07% of visitors. We tested K-Means: with two groups it swallowed the bots into the humans, and with eight it shattered them into pieces. GMM lets each group have its own size and shape, so a small group survives. We also tried Holt-Winters forecasting, and it lost to a simple baseline.

> **Note — K-Means:** the most common clustering method; splits data into k round groups of roughly similar spread.

> **Note — Gaussian Mixture Model (GMM):** models the data as a mix of several bell-curve-shaped groups. Each group can be stretched or small, and each visitor gets a probability of belonging to each group.

> **Note — Holt-Winters:** a forecasting method for time series with trend and weekly patterns. Idea was: predict traffic, and flag big misses as anomalies. A "seasonal-naive" baseline ("same as last week") beat it: 21.2% error vs 21.9%.

Handoff: So that is what we built and why. Kushal will now show you how we built it.

Speaker 2 — Kushal: coding and implementation (~2:30)

The platform

Thank you, Kriti. Our system runs on a real big data stack. We use Hadoop's file system, HDFS, for storage, with one NameNode and two DataNodes, and every file is stored twice. Processing is done with Apache Spark 3.5, using its machine-learning library MLlib. Everything runs inside Docker containers, started with a single command.

> **Note — Hadoop / HDFS:** Hadoop Distributed File System. Splits big files into blocks and spreads them across several machines. The *NameNode* keeps the index of where each block is; *DataNodes* store the actual blocks. "Replication factor 2" = every block is kept on two DataNodes, so losing one doesn't lose data.

> **Note — Apache Spark:** an engine that processes big data in parallel across many machines, in memory, so it's much faster than older Hadoop MapReduce. *PySpark* = using Spark from Python.

> **Note — Spark MLlib:** Spark's built-in machine-learning library. Our MLP and GMM both come from here.

> **Note — Docker / Docker Compose:** Docker packages software into "containers" that run the same on any computer. Docker Compose starts several containers together — here, the whole Hadoop + Spark cluster — with one command.

> **Note — JDK 11:** the Java runtime. Hadoop and Spark are written in Java/Scala, so they need it.

The classification pipeline

The pipeline has six stages. One: the labelled data is copied into HDFS and never edited. Two: Spark reads it and cleans the labels. Three: every request is turned into 42 numbers. Four: the result is saved as Parquet, so features are computed once, not every time. Five: we standardise and train. Six: Spark Structured Streaming scores live traffic every five seconds.

> **Note — Parquet:** a column-based file format. Much smaller and faster to read than CSV. Ours uses Snappy compression and is split into folders by date, so a query for one day only reads that day's folder.

> **Note — Spark Structured Streaming:** Spark's way of handling live data. Instead of one big batch, it processes new data in small "micro-batches" — ours every 5 seconds. *Checkpointed* = it saves its progress, so after a crash it continues where it stopped.

The 42 features

The key design choice is the features. We split them into two groups on purpose. 29 are structural: the length of the request, its entropy, the fraction of punctuation, the number of encoded characters. These never look for any word. The other 13 count known attack words, like "select" or "script" — which is what a signature scanner does.

> **Note — Shannon entropy:** measures how random or mixed-up the characters are. Normal text like "campello" has low entropy; attack code like `%27%3B--` has high entropy.

> **Note — %XX escapes / URL-encoding:** special characters in a URL are written as % plus a code, e.g. %27 is a single quote. Attackers use lots of these.

Why split them? Because a model that is given a list of attack words is really just a signature scanner in disguise. When we removed all 13 keyword features, the score dropped by only 0.0002 and the model became twice as fast. That means the model detects attacks by their shape, not their words — which is exactly what lets it catch new attacks.

The features are computed by a Python function that Spark runs on every worker, called a UDF.

> **Note — UDF (User-Defined Function):** your own Python function that Spark runs on every row, in parallel, across all workers. We had to ship our `features.py` file to every worker with `addPyFile`, otherwise they crash with a "module not found" error.

Training

Before training, every feature is standardised to mean zero and spread one. This is required for a neural network: otherwise request length, which can be in the hundreds, would drown out ratios between zero and one. The split is stratified, because command injection has only 89 rows in total and a random split could leave almost none in the test set.

> **Note — StandardScaler / standardising:** rescales each feature so its average is 0 and its typical spread is 1. Puts all features on the same scale.

> **Note — Stratified split:** each class is split 80/20 separately, so rare classes appear in both the training and test sets in the right proportion.

> **Note — Pipeline (in code):** chains the steps — assemble features → scale → MLP — so the exact same steps run at training and at prediction time.

The clustering pipeline

For clustering we use the NASA web server logs, about 3.4 million lines. First we group requests into sessions: if a visitor is quiet for more than 30 minutes, the next request starts a new visit. Then each visitor gets nine behaviour features — such as the share of HEAD requests, the median gap between requests, the share of images, and how regular their timing is. We apply a log transform so one very busy visitor does not dominate, then standardise everything, and fit the Gaussian Mixture Model with three groups.

> **Note — NASA-HTTP dataset:** two months of real web logs from NASA's Kennedy Space Center server in 1995. It has no labels, which is why it suits clustering.

> **Note — Sessionisation:** grouping one visitor's requests into separate visits. Done with a Spark *window function*, which looks at each request's previous request for the same host.

> **Note — HEAD request:** asks the server only for a page's details (size, last changed), not the page itself. Crawlers use it to check if a page changed.

> **Note — log1p transform:** log(1 + x). Squashes very large numbers so a host with 50,000 requests doesn't overpower everyone else.

The important point is that the bot group is picked out from its own centre values — high HEAD ratio, long regular gaps, few images. We never told the model what a bot is.

> **Note — Centroid:** the average point of a cluster — e.g. "the typical member of group 2 makes 6.78% HEAD requests".

Both tracks have small extras: eleven Spark SQL reports on the same data, and a results website.

> **Note — Spark SQL:** lets you query big data with normal SQL (SELECT … FROM … WHERE …) and Spark runs it in parallel.

Handoff: That is how the system works. Gargi will now show you what it found — including a result we did not expect.

Speaker 3 — Gargi: results and inference (~2:45)

Classification results

Thank you, Kushal. Our training data is the HTTP Params Dataset, about 31,000 requests. 62% are normal and 35% are SQL injection; command injection is only 89 rows.

On the test set, the Multilayer Perceptron scored a macro-F1 of 0.9984. It caught 100% of normal requests, 99.9% of SQL injection and 99.1% of cross-site scripting. Path traversal was 93.2% and command injection 94.7% — but command injection has only 19 test rows, so one mistake changes that number by five points. We do not treat it as reliable.

> **Note — Recall:** of all the real attacks of a type, what percentage the model caught.

We use macro-F1, not accuracy, on purpose. A model that never detects command injection would still get about 96% accuracy, because the data is mostly normal and SQL injection. Macro-F1 makes every class count equally.

> **Note — Accuracy:** simply the percentage of all predictions that were right. Misleading when some classes are rare.

The twist

99.84% sounds excellent. But it made us suspicious. In our training data, the normal requests came from one real website, while every attack was generated by hacking tools like sqlmap. Maybe the model had just learned to recognise those tools.

> **Note — sqlmap, xssya, Vega, FuzzDB:** automatic security-testing tools that generate attack strings. Our training attacks came from these, not from real attackers.

So we tested it on a completely separate dataset, Fwaf — 1.3 million requests collected by different people for a different project. The result was a disaster. The model falsely flagged 92% of normal traffic as attacks.

> **Note — Fwaf:** a public dataset of real-looking URLs labelled attack or benign. We used it only for testing, never training.

> **Note — False positive:** normal traffic wrongly flagged as an attack. A 92% rate makes a system useless — it would block almost everyone.

Why it happened

First we checked the obvious excuse — maybe Fwaf's labels were wrong. Only 0.04% of its normal lines actually contained an attack, far too few.

The real reason is simple. In training, normal data looked like "campello, el" or "40184" — almost no punctuation. Every attack was full of quotes, brackets and semicolons. So the model learned one shortcut: lots of punctuation means attack. But real website addresses, like a search page with question marks and ampersands, are also full of punctuation and completely harmless.

The same failure happened in three unrelated model types — the neural network, factorization machines, and random forest. When a problem survives changing the algorithm, it is not the algorithm. It is the data.

The fix

So we fixed the data, not the model. We split Fwaf in half. We added realistic normal traffic from half A to training, and tested only on half B, which neither model ever saw. False positives fell from 92% to 0.74%. Precision rose to 0.979, F1 on unseen traffic rose from

0.428 to 0.931, and detection stayed at 88.8%. The lesson: the normal examples you train on must look like the traffic the system will really see.

> **Note — Precision:** of everything the model flagged as an attack, what share really was one. 0.979 = about 98% of alerts are real.

> **Note — Detection rate:** same idea as recall — the share of real attacks caught.

Clustering results

For clustering, we ran the model on 400,000 NASA log lines. We kept 4,888 visitors with at least 20 requests. The model found three groups, and one of them, 110 visitors, behaves like automated bots.

One feature separates them clearly: HEAD requests. In the bot group they are 6.78% of traffic. In the two human groups they are 0.05% and 0.01% — about a hundred times less. A HEAD request checks if a page changed without downloading it. Crawlers do this; people reading a website never do. The bot group also fetches fewer images, because a bot wants data, not a nicely drawn page. Nobody told the model about HEAD requests — it discovered this on its own.

> **Note — Silhouette score (0.3465):** measures how well separated the clusters are, from −1 to 1. Higher is better; ours shows a reasonable separation.

> **Note — Crawler / bot:** an automated program that visits websites, e.g. a search engine indexing pages.

Validation and limits

To check this, we held out visitors who asked for robots.txt — crawlers announcing themselves. But most crawlers never ask for it, so this check can only give a lower bound. In this run there was only one such crawler, too few to quote a percentage. Also, clustering finds groups, not intent — the bot group includes harmless search engines. Its output is a list for a human to review, not a verdict.

> **Note — robots.txt:** a file on a website that tells crawlers which pages they may visit. Well-behaved bots request it first.

Other limits we report openly: NASA logs have no response-time field, four days are missing from them, and our distributed system runs on one laptop.

Closing

To finish: our biggest weakness is that the attacks were generated by tools, not captured from real attackers — and that is exactly what caused the 92%. Our next step would be to capture real attack traffic, and add Kafka for production-grade streaming. But finding that flaw in our own result, and fixing it, is the real outcome of this project. Thank you.

> **Note — Kafka:** a system for moving live data streams between programs at large scale. We read from files instead to save memory on a laptop; swapping in Kafka is a one-line change.

Q&A cheat sheet

Each question goes to the speaker who covered that part, so nobody talks over each other.

Question	Who answers	Short answer
Why not Random Forest? It scored almost the same.	Kriti	Faculty excluded common models, and the MLP beat it anyway on score (0.9984 vs 0.9981) and speed (20.3 vs 49.1 ms).
Why GMM over K-Means?	Kriti	Bots are ~0.07% of hosts. K-Means k=2 swallowed them (worse than random); k=8 shattered them (15% recall). GMM allows small, oddly shaped groups.
Why k = 3?	Kriti	Fewer groups absorbed the bots, more groups split them; 3 isolated them with silhouette 0.3465.
Is this really big data on one laptop?	Kushal	The architecture is genuinely distributed (HDFS with 2 DataNodes, Spark master + worker); the hardware is one machine.
Why not TF-IDF?	Kushal	It creates tens of thousands of sparse columns — the "feature-dimension overhead" the problem statement asks us to avoid.

Question	Who answers	Short answer
The keyword-removal test was on a tree model — why?	Kushal	It was the common baseline so both feature sets could be compared fairly; the tree model itself is not deployed.
Why scale the features?	Kushal	Neural networks and clustering are scale-sensitive; without it, large-valued features like length dominate.
Isn't 99.84% good?	Gargi	It was measuring the gap between two data sources. On independent data, false positives were 92%.
How do you know it's the data, not the model?	Gargi	The same failure appeared in three unrelated model families, and only changing the training data fixed it.
Could Fwaf's labels just be wrong?	Gargi	Only 0.04% of its normal lines contain a real attack signature — far too few to explain 92%.
What is your recall for bots?	Gargi	Only one held-out robots.txt crawler in this run — too few to quote a rate. It's a lower bound at best.

Before you present

- [ ] Replace the GitHub placeholder `https://github.com/[username]/log-analytics-bde` in report sections 1.6 and 2.6 and on the last slide — it is worth 1 of the 6 document marks.
- [ ] Don't quote the "44.4× lift" for clustering aloud — it comes from a single held-out crawler.
- [ ] Each speaker rehearses once with a timer; Gargi's part is the longest and carries the main story.
- [ ] Practise the two handoff lines so the transitions are smooth.